Module 4

Interrupt Programming: Basics of Interrupts, 8051 Interrupts, Programming Timer Interrupts, Programming Serial Communication Interrupts, Interrupt Priority in 8051(Assembly Language only) (Text book 2- 3.6, Text book 1- 11.1,11.2,11.4, 11.5)

BASICS OF INTERRUPTS.

An interrupt is an external or internal event that interrupts the microcontroller to inform it that a device needs its service.

A single microcontroller can serve several devices by two ways

(i)Interrupts

- Whenever any device needs its service, the device notifies the microcontroller by sending it an interrupt signal
- Upon receiving an interrupt signal, the microcontroller interrupts whatever it is doing and serves the device
- The program which is associated with the interrupt is called the interrupt service routine (ISR) or interrupt handler

(ii) Polling

- The microcontroller continuously monitors the status of a given device. When the conditions met, it performs the service. After that, it moves on to monitor the next device until every device is serviced
- Polling can monitor the status of several devices and serve each of them as certain conditions are met
- The polling method is not efficient, since it wastes much of the microcontroller's time by polling devices that do not need service, ex.target: JNB TFx,target

The advantage of interrupts is that the microcontroller can serve many devices (not all at the same time) based on some defined priority.

- Each devices can get the attention of the microcontroller based on the assigned priority. For the polling method, it is not possible to assign priority since it checks all devices in a round-robin fashion
- The microcontroller can also ignore (mask) a device request for service. This is not possible for the polling method

For every interrupt, there must be an interrupt service routine (ISR), or interrupt handler

When an interrupt is invoked, the microcontroller runs the interrupt service routine. For every interrupt, there is a fixed location in memory that holds the

address of its ISR. The group of memory locations set aside to hold the addresses of ISRs is called interrupt vector table.

Steps involved in executing or handling an Interrupt in 8051

Upon activation of an interrupt, the microcontroller goes through the following steps

1.It finishes the instruction it is executing and saves the address of the next instruction (PC) on the stack 2.It also saves the current status of all the interrupts internally (i.e: not on the stack)

3.It jumps to a fixed location in memory, called the interrupt vector table, that holds the address of the ISR

4. The microcontroller gets the address of the ISR from the interrupt vector table and jumps to it.ƒIt starts to execute the interrupt service subroutine until it reaches the last instruction of the subroutine which is RETI (return from interrupt)

5. Upon executing the RETI instruction, the microcontroller returns to the place where it was interrupted ƒFirst, it gets the program counter (PC) address from the stack by popping the top two bytes of the stack into the PC ƒThen it starts to execute from that address.

8051 Interrupts

Six interrupts of 8051 are allocated as follows

1. Reset –power-up reset

2. Two interrupts are set aside for the timers: one for timer 0 and one for timer 1

3. Two interrupts are set aside for hardware external interrupts ƒP3.2 and P3.3 are for the external hardware interrupts INT0 (or EX1), and INT1 (or EX2)

4. Serial communication has a single interrupt that belongs to both receive and transfer.

Interrupt vector table

Interrupt	ROM Location (hex)	Pin
Reset	0000	9
External HW (INT0)	0003	P3.2 (12)
Timer 0 (TF0)	000B	
External HW (INT1)	0013	P3.3 (13)
Timer 1 (TF1)	001B	
Serial COM (RI and TI)	0023	

Upon reset, all interrupts are disabled (masked), meaning that none will be responded to by the microcontroller if they are activated ‰

The interrupts must be enabled by software in order for the microcontroller to respond to them. There is a register called IE (interrupt enable) that is responsible for enabling (unmasking) and disabling masking) the interrupts.

IE Register – Interrupt Enable Register

7	6	5	4	3	2	1	0
EA	X	X	ES	ET1	EX1	ET0	EX0

- EX0- Enable/Disable – External Interrupt 0
- ET0- Enable/Disable – Timer 0 overflow Interrupt
- EX1- Enable/Disable – External Interrupt 1
- ET1- Enable/Disable – Timer 1 overflow Interrupt
- ES -Enable/Disable – Serial port Interrupt
- EA - Enable/Disable – All interrupts

Examples –

-Disable all interrupts

```
CLR IE.7
```

To enable an interrupt, we take the following steps:

1.Bit D7 of the IE register (EA) must be set to high to allow the rest of register to take effect

2. The value of EA. If EA = 1, interrupts are enabled and will be responded to if their corresponding bits in IE are high

3. If EA = 0, no interrupt will be responded to, even if the associated bit in the IE register is high.

Example:

Show the instructions to (a) enable the serial interrupt, timer 0 interrupt, and external hardware interrupt 1 (EX1),and (b) disable (mask) the timer 0 interrupt, then (c) show how to disable all the interrupts with a single instruction.

Solution:

(a)MOV IE,#10010110B ;enable serial, ;timer 0, EX1

Another way to perform the same manipulation is

SETB IE.7 ;EA=1, global enable

SETB IE.4 ;enable serial interrupt

SETB IE.1 ;enable Timer 0 interrupt

SETB IE.2 ;enable EX1

(b)CLR IE.1 ;mask (disable) timer 0 ;interrupt only

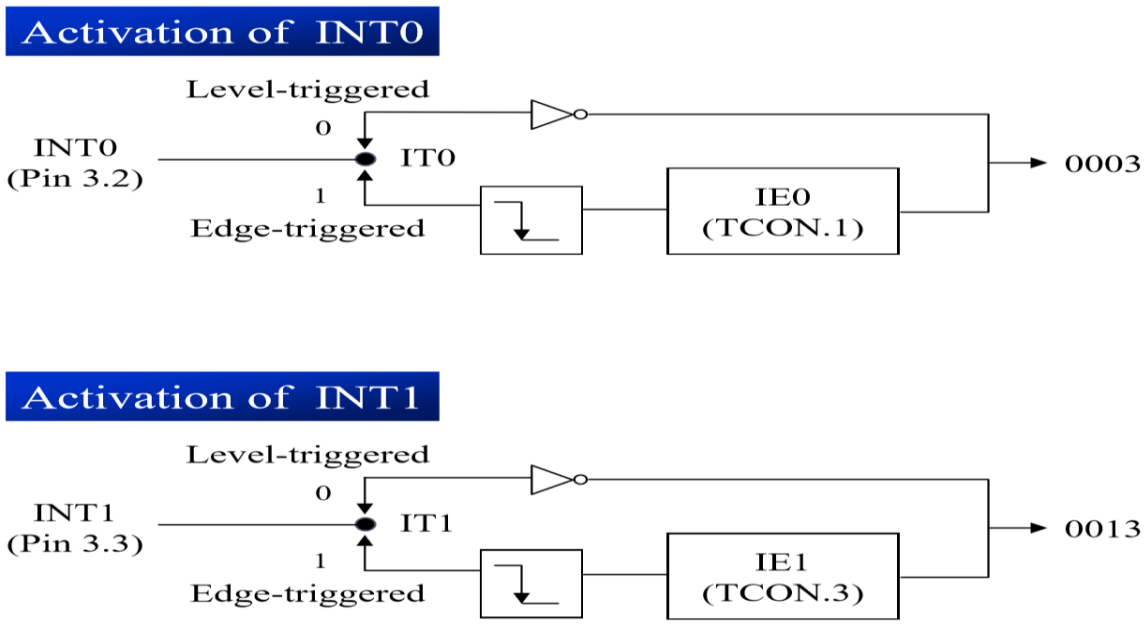
(c)CLR IE.7 ;disable all interrupts

PROGRAMMING OF EXTERNAL INTERRUPTS

The 8051 has two external hardware interrupts. Pin 12 (P3.2) and pin 13 (P3.3) of the 8051, designated as INT0 and INT1, are used as external hardware interrupts.

The interrupt vector table locations 0003H and 0013H are set aside for INT0 and INT1.There are two activation levels for the external hardware interrupts ƒ

(i) Level trigged ƒ
(ii) Edge trigged



LEVEL TRIGGERED INTERRUPTS

In the level-triggered mode, INT0 and INT1 pins are normally high. If a low-level signal is applied to them, it triggers the interrupt ,Then the microcontroller stops whatever it is doing and jumps to the interrupt vector table to service that interrupt. The low-level signal at the INT pin must be removed before the execution of the last instruction of the ISR, RETI; otherwise, another interrupt will be generated. This is called a level-triggered or level activated interrupt and is the default mode upon reset of the 8051.

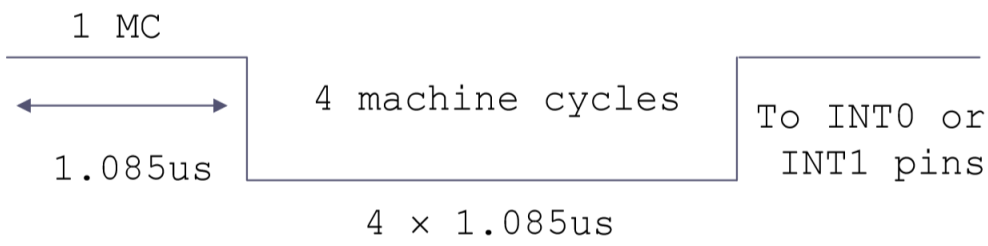
Pins P3.2 and P3.3 are used for normal I/O unless the INT0 and INT1 bits in the IE register are enabled. After the hardware interrupts in the IE register are enabled, the controller keeps sampling the INTn pin for a low-level signal once each machine cycle.

According to one manufacturer's data sheet, the pin must be held in a low state until the start of the execution of ISR.If the INTn pin is brought back to a logic high before the start of the execution of ISR there will be no interrupt.

If INTn pin is left at a logic low after the RETI instruction of the ISR, another interrupt will be activated after one instruction is executed.

To ensure the activation of the hardware interrupt at the INTnpin, make sure that the duration of the low-level signal is around 4 machine cycles, but no more.

This is due to the fact that the level-triggered interrupt is not latched.ƒThus the pin must be held in a low state until the start of the ISR execution.



EDGE TRIGGERED INTERRUPTS

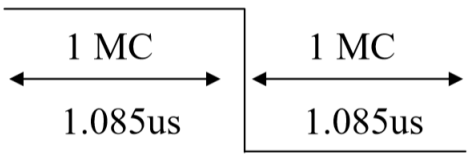
To make INT0 and INT1 edge triggered interrupts, we must program the bits of the TCON register. The TCON register holds, among other bits, the IT0 and IT1 flag bits that determine level-or edge-triggered mode of the hardware interrupt.

IT0 and IT1 are bits D0 and D2 of the TCON register.They are also referred to as TCON.0 and TCON.2 since the TCON register is bit addressable.

In edge-triggered interrupts, the external source must be held high for at least one machine cycle, and then held low for at least one machine cycle.

The falling edge of pins INT0 and INT1 are latched by the 8051 and are held by the TCON.1 (IE0)and TCON.3(IE1) bits of TCON register. Function as interrupt-in-service flags. It indicates that the interrupt is being serviced now and on this INTnpin, and no new interrupt will be responded to until this service is finished.

Minimum pulse duration to detect edge-triggered interrupts XTAL=11.0592MHz



TCON (Timer/Counter) Register (Bit-addressable) (cont')

IE1	TCON.3	External interrupt 1 edge flag. Set by CPU when the external interrupt edge (H-to-L transition) is detected. Cleared by CPU when the interrupt is processed
IT1	TCON.2	Interrupt 1 type control bit. Set/cleared by software to specify falling edge/low-level triggered external interrupt
IE0	TCON.1	External interrupt 0 edge flag. Set by CPU when the external interrupt edge (H-to-L transition) is detected. Cleared by CPU when the interrupt is processed
IT0	TCON.0	Interrupt 0 type control bit. Set/cleared by software to specify falling edge/low-level triggered external interrupt

Regarding the IE0 and IE1 bits in the TCON register, the following two points must be emphasized

When the ISRsare finished (that is, upon execution of RETI), these bits (TCON.1 and TCON.3) are cleared, indicating that the interrupt is finished and the 8051 is ready to respond to another interrupt on that pin.

During the time that the interrupt service routine is being executed, the INTnpin is ignored, no matter how many times it makes a high-to-low transition.

RETI clears the corresponding bit in TCON register (TCON.1 or TCON.3). There is no need for instruction CLR TCON.1 before RETI in the ISR associated with INT0.

DIFFERENCE BETWEEN RET AND RETI INSTRUCTIONS

Both perform the same actions of popping off the top two bytes of the stack into the program counter, and making the 8051 return to where it left off.

However, RETI also performs an additional task of clearing the interrupt-in-service flag like (IE0, IE1, TF0, TF1), indicating that the servicing of the interrupt is over and the 8051 now can accept a new interrupt on that pin. If you use RET instead of RETI as the last instruction of the interrupt service routine, you simply block any

new interrupt on that pin after the first interrupt, since the pin status would indicate that the interrupt is still being serviced.

In the cases of TF0, TF1, TCON.1, and TCON.3, they are cleared due to the execution of RETI.

PRIORITIES OF INTERRUPTS

When the 8051 is powered up, the priorities are assigned according to the following. In reality, the priority scheme is nothing but an internal polling sequence in which the 8051 polls the interrupts in the sequence listed and responds accordingly.

Interrupt Priority Upon Reset.

Highest To Lowest Priority	
External Interrupt 0	(INT0)
Timer Interrupt 0	(TF0)
External Interrupt 1	(INT1)
Timer Interrupt 1	(TF1)
Serial Communication	(RI + TI)

We can alter the sequence of interrupt priority by assigning a higher priority to any one of the interrupts by programming a register called IP (interrupt priority).

To give a higher priority to any of the interrupts, we make the corresponding bit in the IP register high. When two or more interrupt bits in the IP register are set to high, the interrupts that have a higher priority than others, are serviced according to the default priority sequence.

Interrupt Priority Register (Bit-addressable)

D7							D0
--	--	PT2	PS	PT1	PX1	PT0	PX0

--	IP.7	Reserved
--	IP.6	Reserved
PT2	IP.5	Timer 2 interrupt priority bit (8052 only)
PS	IP.4	Serial port interrupt priority bit
PT1	IP.3	Timer 1 interrupt priority bit
PX1	IP.2	External interrupt 1 priority bit
PT0	IP.1	Timer 0 interrupt priority bit
PX0	IP.0	External interrupt 0 priority bit

Priority bit=1 assigns high priority

Priority bit=0 assigns low priority

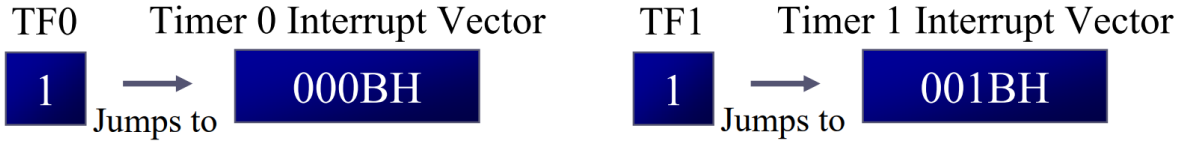
Assume that the INT1 pin is connected to a switch that is normally high. Whenever it goes low, it should turn on an LED. The LED is connected to P1.3 and is normally off. When it is turned on it should stay on for a fraction of a second. As long as the switch is pressed low, the LED should stay on.

```
        ORG  0000H
        SJMP MAIN ;by-pass interrupt
;vector table ;--ISR for INT1 to turn on LED
        ORG  0013H     ;INT1 ISR
        SETB P1.3      ;turn on LED
        MOV  R3,#255
 BACK: DJNZ R3,BACK    ;keep LED on for a while
        CLR  P1.3      ;turn off the LED
        RETI           ;return from ISR
;--MAIN program for initialization
        ORG  30H
MAIN: MOV  IE,#10000100B ;enable external INT 1
```

```
HERE: SJMP HERE     ;stay here until get interrupted
END
```

programming of timer interrupts.

The timer overflow flag (TF) is raised when the timer rolls over from FFFFh to 0000h(In mode 1) and FFh to 00h (In mode 2). In polling TF, we have to wait until the TF is raised. The microcontroller is tied down while waiting for TF to be raised, and cannot do anything else. If the timer interrupt in the IE register is enabled, whenever the timer rolls over, TF is raised. This avoids tying down the controller. The microcontroller is interrupted in whatever it is doing, and jumps to the interrupt vector address 000Bh(Timer0 interrupt) or 001Bh (Timer1 interrupt) in interrupt vector table to service the ISR. In this way, the microcontroller can do other task until it is notified that the timer has rolled over after some defined period of time.



PROGRAMMING TIMER INTERRUPTS using C

The 8051 compiler have extensive support for the interrupts. They assign a unique number to each of the 8051 interrupts.

Interrupt	Name	Numbers
External Interrupt 0	(INT0)	0
Timer Interrupt 0	(TF0)	1
External Interrupt 1	(INT1)	2
Timer Interrupt 1	(TF1)	3
Serial Communication	(RI + TI)	4
Timer 2 (8052 only)	(TF2)	5

PROGRAM1

Write a program that continuously get 8-bit data from P0 and sends it to P1 while simultaneously creating a square wave of 200 µs period on pin P2.1. Use timer 0 to create the square wave. Assume that XTAL = 11.0592 MHz.

Solution:

We will use timer 0 in mode 2 (auto reload). Period of square wave =200µs.
Ton=Toff =100 µs

TH0 = 100 µs /1.085 us = 92

```
 ;--upon wake-up go to main, avoid using ;memory allocated to Interrupt Vector Table
        ORG  0000H
        SJMP MAIN  ;by-pass interrupt vector table ;
;--ISR for timer 0 to generate square wave
        ORG  000BH ;Timer 0 interrupt vector table
        CPL  P2.1  ;toggle P2.1 pin
        RETI       ;return from ISR
;--The main program for initialization
        ORG  0030H    ;after vector table space
MAIN: MOV  TMOD,#02H ;Timer 0, mode 2
        MOV  P0,#0FFH ;make P0 an input port
        MOV  TH0,#-92 ;TH0=A4H for -92
        MOV  IE,#82H  ;IE=10000010 (bin) enable ;Timer 0
        SETB TR0      ;Start Timer 0
BACK: MOV  A,P0     ;get data from P0
        MOV  P1,A     ;issue it to P1
        SJMP BACK     ;keep doing it loop ;unless interrupted by TF0
        END
```

8051 C program

```
# include <reg51.h>
Sfr in = P0;
Sfr out = P1;
Sbit wave = P2.1;
Void main()
{
TMOD=0X02;
IE=0X82;
TH0=0XA4;
in=0XFF;
TR0=1;
While(1)
{
out=in;
}}
Void timer0ISR() interrupt 1
{
```

```
Wave=~wave;
}
```

PROGRAM 2

Write a C program that continuously gets a single bit of data from P1.7 and sends it to P1.0, while simultaneously creating a square wave of 200 µs period on pin P2.5. Use Timer 1 to create the square wave. Assume that XTAL = 11.0592 MHz.

Solution:

We will use timer 0 in mode 2 (auto reload). Period of square wave =200µs.
Ton=Toff =100 µs

N = 100 µs /1.085 us = 92 and TH1 = 256 -92 = 164 or A4H

```
#include <reg51.h>
Sbit IN   =P1^7;
Sbit OUT =P1^0;
sbitWAVE =P2^5;
void timer1ISR( ) interrupt 3
 {
WAVE=~WAVE;  //toggle pin
 }
void main()
{
 IN=1;     //make P1.7 as input
TMOD=0x20;
 TH1=0xA4; //TH0=-92
IE=0x88;//enable interrupt for timer 1
while (1)
{
OUT=IN; //send data from P1.7 to P1.0
}}
```

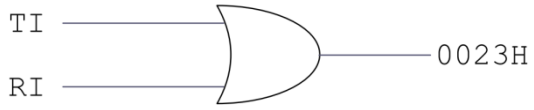
Serial Communication Interrupts programming in 8051

TI (transfer interrupt) is raised when the stop bit is transferred indicating that the SBUF register is ready to transfer the next byte

RI (received interrupt) is raised when the stop bit is received indicating that the received byte needs to be picked up before it is lost (overrun) by new incoming serial data

In the 8051 there is only one interrupt set aside for serial communication, used for both sending and receiving data. If the interrupt bit in the IE register (IE.4) is

enabled, when RI or TI is raised the 8051 gets interrupted and jumps to memory location 0023H to execute the ISR. In that ISR we must examine the TI and RI flags to see which one caused the interrupt and respond accordingly.



Serial interrupt is invoked by TI or RI flags

Write a program in which the 8051 reads data from P1 and writes it to P2 continuously while giving a copy of it to the serial COM port to be transferred serially. Assume that XTAL=11.0592. Set the baud rate at 9600.

Solution:

```
        ORG  0000H
        LJMP MAIN
        ORG  23H
        LJMP SERIAL    ;jump to serial intISR
        ORG  30H
MAIN: MOV  P1,#0FFH  ;make P1 an input port
        MOV  TMOD,#20H ;timer 1, auto reload
        MOV  TH1,#0FDH ;9600 baud rate
        MOV  SCON,#50H ;8-bit,1 stop, renenabled
        MOV  IE,10010000B ;enable serial int.
        SETB TR1       ;start timer 1
BACK: MOV  A,P1      ;read data from port 1
        MOV  SBUF,A    ;give a copy to SBUF
        MOV  P2,A      ;send it to P2
        SJMP BACK      ;stay in loop indefinitely
        ----------------SERIAL PORT ISR
        ORG 100H
SERIAL: JB  TI,TRANS      ;jump if TI is high
        MOV A,SBUF         ;otherwise due to receive
        CLR RI             ;clear RI since CPU doesn't
        RETI               ;return from ISR
TRANS: CLR TI              ;clear TI since CPU doesn't
        RETI               ;return from ISR
        END
```

The moment a byte is written into SBUF it is framed and transferred serially. As a result, when the last bit (stop bit) is transferred the TI is raised, and that causes the serial interrupt to be invoked since the corresponding bit in the IE register is high. In the serial ISR,we check for both TI and RI since both could have invoked interrupt.

8051 C program

```
# include <reg51.h>
sfr in = P1;
```

```
sfr out = P2;
Void main()
{
P1=0XFF;
TMOD=0X20;
SCON=0X50;
IE=0X90;
TH1=0XFD;
TR1=1;
While(1)
{
ACC=in
SBUF=ACC;
Out=ACC;
}
}
void serialISR ( ) interrupt 4
{
If (TI==1)
{
TI=0;
}
else
{
RI=0;
ACC=SBUF;
}
}
```